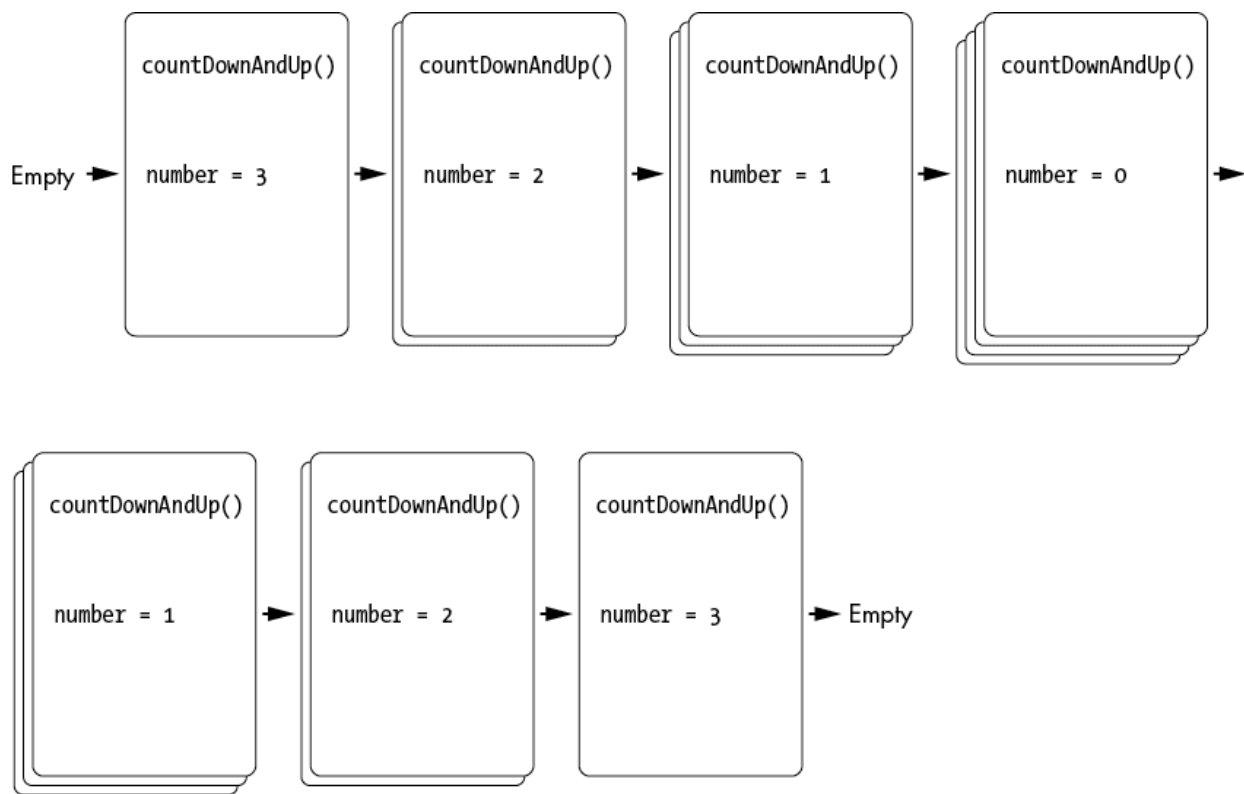


numbers to appear. Once `countDownAndUp(0)` is called, the base case is reached ❷, and no more recursive calls are made. However, this isn't the end of our program! When the base case is reached, the local `number` variable is 0. But when that base case returns, and the frame is popped off the call stack, the frame under it has its own local `number` variable, with the same 1 value it's always had. As the execution returns back to the previous frames in the call stack, the code *after* the recursive call is executed ❹. This is what causes the count up of numbers to appear. [Figure 1-9](#) shows the state of the call stack as `countDownAndUp()` is recursively called and then returns.



[Figure 1-9](#): The call stack keeping track of the values in the `number` local variable for each function call

The fact that the code doesn't stop immediately when the base case is reached will be important to keep in mind for the factorial calculation in the next chapter. Remember, any code after the recursive case will still have to run.

At this point, you might be thinking that the recursive `countDownAndUp()` function is overengineered and difficult to follow. Why not, instead, use an iterative solution to print numbers? An *iterative* approach, which uses loops